AI DD

AI-Powered M&A Due Diligence Platform

Complete Technical Study Guide

Combined: Main Branch + Version 1.0

Architecture | Implementation | Algorithms | Interview Preparation

Backend: Python 3.12 + FastAPI + SQLAlchemy 2.0 (async) + Celery + Redis

AI/LLM: Groq API (Llama 3.3 70B) | Google Vertex AI (Gemini 1.5 Pro)

Search: PostgreSQL FTS (tsvector) | pgvector (768-dim cosine similarity)

OCR: pymupdf + pytesseract + pypdf + python-docx + openpyxl

Frontend: React 19 + TypeScript + Tailwind CSS + React Query + Zustand

Auth: JWT + bcrypt + 2FA/TOTP + token blacklist + GDPR deletion

Infrastructure: Docker Compose | Google Cloud Run (europe-west3)

11 Modules | 60+ API Endpoints | 17+ Database Tables | 6 Alembic Migrations

April 2026 | *CONFIDENTIAL*

1. Project Overview

The 30-Second Pitch

> *AI DD is an AI-powered M&A Due Diligence platform for advisors and investors. It automates the due diligence process from document collection through AI-assisted analysis to report generation. The core innovation is a RAG-based multi-agent system where four specialized AI agents (Legal, Tax, Finance, Planning) analyze uploaded documents and produce structured findings -- all grounded in actual evidence. Every AI output requires human approval before reaching external parties.*

Core Value Proposition

- Faster due diligence through AI-powered document processing and structured workflows
- No hallucination by design -- RAG architecture ensures AI only works with uploaded data
- Active gap identification -- proactively identifies missing information via Smart Question Engine
- Full human control -- no external communication without explicit approval (HITL principle)
- GDPR compliant -- EU data residency, audit trail, right to deletion

Two Development Branches

Feature	Main Branch	Version 1.0
LLM Provider	Google Vertex AI (Gemini)	Groq API (Llama 3.3 70B)
RAG Search	pgvector cosine similarity	PostgreSQL FTS (tsvector)
Task Queue	BackgroundTasks	Celery + Redis (configured)
Auth	JWT + 2FA + blacklist + GDPR	JWT + basic (no 2FA)
DMS	Versioning + tagging + search	Basic upload + OCR
Finance	SKR detection + German nums	pandas-based import
Reports	Not built	Word .docx generation
Audit	Not built	Middleware logging
OCR	pypdf + python-docx	pymupdf + pytesseract
Migrations	6 Alembic files	create_all() only
Package Mgr	pip + requirements.txt	uv + pyproject.toml

2. System Architecture

Service Topology

```
  React Frontend (:5173)  --->  FastAPI Backend (:8000)
                                        |
                            +-----------+-----------+
                            |                       |
                      PostgreSQL 16           Redis 7
                      (:5432/5433)           (:6379/6750)
                      + pgvector               Celery
                      + tsvector FTS            Worker
```

Full Tech Stack

Layer	Technology	Purpose
Backend	FastAPI + Python 3.12	Async REST API
ORM	SQLAlchemy 2.0 (async)	DB abstraction
Database	PostgreSQL 16 + pgvector	Data + vectors + FTS
AI (v1.0)	Groq API (Llama 3.3 70B)	Free-tier LLM
AI (main)	Vertex AI (Gemini 1.5 Pro)	Enterprise LLM
Embeddings	text-embedding-004 (768d)	Main branch only
FTS	tsvector + GIN index	Full-text search
OCR	pymupdf + pytesseract + Pillow	PDF/image extraction
Task Queue	Celery + Redis	Background processing
Frontend	React 19 + TypeScript	SPA
Styling	Tailwind CSS	Dark M&A-grade UI
State	React Query + Zustand	Server + client state
Auth	JWT (HS256) + bcrypt	Tokens + password hashing
Reports	python-docx + openpyxl	Word/Excel generation

11 Backend Modules

Module	Endpoints	Status	Key Feature
Auth	8-14	100%	JWT + roles + 2FA (main) / basic (v1.0)
Projects	5-7	100%	CRUD + members + access control
DMS	5-12	75-90%	Upload + OCR + versioning (main)
Agent	4	85%	4 agents + RAG + Groq/Vertex
Planning	6-9	100%	5-phase workflow + AI generation
Finance	5-14	100%	Import + variance + SKR (main)
Report	5	100%*	Word .docx (v1.0 only)
Audit	2	100%*	Middleware logging (v1.0 only)
Knowledge	2	10%	Scaffolded, no logic
Self-Improve	3	10%	Scaffolded, no logic
OCR	internal	100%	pymupdf + pytesseract

3. Authentication & Authorization

JWT Token Structure

```
Header:  {alg: HS256, typ: JWT}
Payload: {sub: user-uuid, role: admin, type: access,
          jti: unique-id (main only), exp: timestamp}
Signature: HMAC-SHA256(header.payload, SECRET_KEY)

Access token:  60 minutes  (short-lived)
Refresh token: 30 days     (for session persistence)
```

Token Validation Chain

```
Browser: Authorization: Bearer eyJ...
  -> HTTPBearer extracts token
  -> jwt.decode() verifies signature + expiry
  -> Check type == 'access'
  -> (Main) Check jti NOT in token_blacklist
  -> Load User from DB by payload['sub']
  -> Check user.is_active == True
  -> Return User object to endpoint
```

5-Role Permission Matrix

Role	Projects	Docs	Analysis	Reports	Admin
Admin	Full	Full	Run+Review	Generate	Full
Lead Advisor	Create	Full	Run+Review	Generate	Audit
Team Advisor	View own	Upload	Run+Review	Drafts	None
Seller	View own	Upload	None	None	None
Buyer	Approved	Approved	None	Finalized	None

Main Branch Additions (not in v1.0)

- Token blacklist table -- proper logout (blacklists jti)
- 2FA/TOTP -- pure Python implementation (HMAC-SHA1, RFC 6238)
- Password reset -- one-time tokens with 1-hour expiry
- GDPR deletion -- cascades across all tables
- Settings page -- change password, manage 2FA, delete account

Password Hashing: bcrypt

```
bcrypt.hashpw(password, bcrypt.gensalt())  # ~100ms per hash
# Output: "$2b$12$..." ($12$ = 2^12 = 4096 rounds)
# Attacker brute force: ~10 passwords/sec (vs 10B/sec with SHA256)
```

4. RAG Pipeline (Two Implementations)

Main Branch: pgvector + Vertex AI

```
1. CHUNK: Document -> 1500-char chunks (300 overlap)
2. EMBED: text-embedding-004 -> 768-dim vector per chunk
3. STORE: pgvector column (Vector(768))
4. SEARCH: cosine_distance(query_embedding, chunk_embedding)
5. GENERATE: Top 10 chunks -> Gemini 1.5 Pro -> JSON findings
```

Version 1.0: PostgreSQL FTS + Groq

```
1. CHUNK: Document -> 1500-char chunks (300 overlap)
2. INDEX: to_tsvector('english', chunk_text) -> GIN index
3. SEARCH: plainto_tsquery() + ts_rank() -> top 10 chunks
4. GENERATE: Top 10 chunks -> Groq (Llama 3.3 70B) -> JSON

Key difference: No vector embeddings needed.
PostgreSQL FTS is simpler but less semantic than cosine similarity.
```

Why Two Approaches?

pgvector captures semantic similarity ('company acquired' matches 'business purchased'). FTS matches exact terms only. For M&A due diligence where legal language is precise, FTS works surprisingly well. pgvector is better for fuzzy/conceptual queries.

Chunking Algorithm

```
CHUNK_SIZE = 1500 chars  (~1 paragraph)
CHUNK_OVERLAP = 300 chars (20%)

Chunk 1: [chars 0-1500]
Chunk 2: [chars 1200-2700]  <- 300 char overlap
Chunk 3: [chars 2400-3900]

Overlap ensures passages at boundaries appear in both chunks.
```

4 Specialized Agents

Agent	Focus	Primary Query
Planning	Doc gaps, scope	due diligence checklist scope completeness
Legal	Contracts, IP, litigation	change of control IP compliance litigation
Tax	CIT, VAT, transfer pricing	tax liabilities deferred tax VAT R&D credits
Finance	EBITDA, revenue, WC	EBITDA revenue recognition working capital

Anti-Hallucination Rules

- Layer 1 - RAG: Every answer grounded in uploaded documents with source citations
- Layer 2 - No-Answer: Returns 'insufficient data' instead of guessing
- Layer 3 - Strict Prompts: 'Do NOT flag positive statements as risks'
- Layer 4 - Quality Cap: Maximum 3-5 findings per agent (quality > quantity)

5. Planning Module (The USP)

5-Phase Interactive Workflow

Phase 1: Basic Data	Company name, legal form, industry, employees, revenue, deal type
Phase 2: Risk Analysis	AI generates 5-12 risk areas based on profile (legal-form, industry, deal-type)
Phase 3: Dialog	AI asks 5-10 targeted follow-up questions (yes/no, text, multi-choice)
Phase 4: Audit Plan	Structured plan: workstreams, scope, focus areas, priorities. Human reviews + approves.
Phase 5: Request List	15-40 items across Legal/Tax/Finance with status + priority + Excel export

Risk Analysis Logic (Main Branch)

```
# Rule-based heuristics (no LLM needed):
GmbH/AG       -> Corporate Governance risk (high)
KG            -> Partnership Structure risk (high)
Tech/Software -> IP Ownership (critical), IFRS 15 (high)
Manufacturing -> Environmental Liability (high)
Share deal    -> Tax Loss Carryforwards s.8c KStG (high)
Asset deal    -> Asset Transfer Tax (high)
Always        -> Change of Control, Employment, Working Capital
```

Request List (Standard DD Items)

Spec s.11 defines the knowledge base. Main branch generates 40 items (18 Legal, 10 Tax, 12 Finance). V1.0 generates 15 template items.

6. Financial Analysis

Main Branch: Intelligent Import Pipeline

```
1. Structure Detection: Scan 20 rows for headers, account codes, periods
2. SKR Detection: Revenue in 4xxx=SKR03, 8xxx=SKR04
3. German Numbers: '1.234,56' -> remove dots -> comma to dot -> Decimal
4. Account Mapping: Lookup ranges in account_mappings table
5. UPSERT: ON CONFLICT (project, account, period) DO UPDATE

Variance Analysis (all in SQL):
  MoM: current month vs prior month per category
  YoY: cumulative YTD comparison
  Significance: >50%/500K=critical, >25%/100K=significant
  Auto-queries: Template-based for significant variances
```

Version 1.0: Pandas-Based Import

```
1. pandas.read_excel() or read_csv()
2. Clean column names, replace NaN
3. Store raw_data as JSON
4. Variance: mean, std, min, max per column
5. Flag significant if variance_pct > 20%
```

SKR03 vs SKR04 (German Chart of Accounts)

Range	SKR03	SKR04
Revenue	4000-4999	8000-8999
Material/COGS	5000-5999	5000-5199
Personnel	6000-6199	6000-6199
Other OpEx	6200-6599	6200-6599
Depreciation	6600-6699	6800-6899

7. Report Generation + Audit Trail

3 Report Types (v1.0)

Type	Content	Format
Detailed Workstream	All findings per workstream, by category	Word .docx
Executive Summary	Critical + high only, editable before export	Word .docx
Consolidated	Cross-workstream, grouped by agent type	Word .docx

Report Pipeline

```
1. Fetch completed AgentRun rows
2. Load AgentFinding rows, deduplicate by title
3. Structure content by report type
4. Append AI Disclaimer (mandatory)
5. On finalize: python-docx creates .docx file
6. Locked (is_finalized=True, immutable)
```

Audit Trail (v1.0)

```
AuditMiddleware on every API request:
  - Captures: method, path, status, IP, user-agent
  - INSERT-ONLY (never UPDATE/DELETE)
  - GDPR tamper-proof design
  - Admins see all logs, Lead Advisors see own only
  - Skips: /health, /docs, /redoc
```

8. Frontend Architecture

Route Map

```
PUBLIC: /, /login, /register, /disclaimer
PROTECTED:
  /dashboard, /projects, /settings
  /projects/:id/documents
  /projects/:id/planning
  /projects/:id/analysis, /analysis/:runId
  /projects/:id/finance
  /projects/:id/reports
  /projects/:id/red-flags
  /projects/:id/settings
  /admin/users (admin only)
  /audit (admin + lead_advisor)
```

State Management

```
React Query (server state):
  - Cache + background refetch + conditional polling
  - Documents: poll every 2s while status='processing'
  - 30-second staleTime, 1 retry

Zustand (client state):
  - user: User | null
  - isLoading: boolean
  - logout(): calls API -> clears localStorage

Axios Interceptor:
  - Request: adds Bearer token from localStorage
  - Response 401: clears tokens -> redirect to /login
```

Permission Hook (v1.0)

```
const p = usePermissions();
p.canCreateProject      // admin, lead_advisor
p.canUploadDocuments    // admin, lead, team, seller
p.canRunAnalysis        // admin, lead, team
p.canManageReports      // admin, lead, team
p.onlyFinalizedReports  // buyer only
p.isReadOnly            // buyer only
```

9. Interview Questions & Answers

Q: Walk me through the system architecture.

> *Monorepo with FastAPI backend and React frontend, containerized with Docker Compose (5 services). Backend has 11 modules, each with own models/schemas/router. PostgreSQL for data + vectors + FTS. Two AI approaches: pgvector + Vertex AI (main) or tsvector + Groq (v1.0). Celery + Redis for background tasks. JWT auth with 5-role hierarchy.*

Q: Why a monolith instead of microservices?

> *For v1 targeting SME deals, a monolith gives faster dev velocity, simpler deployment, easier debugging. Each module is architecturally independent (own models, schemas, router) -- a 'modular monolith' that can split later. The spec calls for microservices; we designed module boundaries to support it without premature complexity.*

Q: Why PostgreSQL FTS instead of pgvector in v1.0?

> *Simpler architecture -- no external embedding service needed. For M&A DD where legal language is precise, keyword-based FTS works well. pgvector adds semantic similarity ('company acquired' = 'business purchased') but requires Vertex AI for embeddings. FTS is self-contained in PostgreSQL.*

Q: How do you prevent AI hallucination?

> *4 layers: 1) RAG -- AI only sees uploaded documents. 2) No-answer policy -- returns 'insufficient data' not guesses. 3) Strict prompts -- 'Do NOT flag positive statements as risks.' 4) Quality cap -- max 3-5 findings per agent, quality > quantity.*

Q: How does your authentication work?

> *JWT with access (60min) + refresh (30d) tokens. bcrypt for password hashing. 5-role hierarchy (admin -> buyer). Project-level access via ProjectMember table. Main branch adds: token blacklist for logout, 2FA/TOTP, password reset, GDPR deletion. Every project-scoped endpoint checks membership first.*

Q: What happens when Groq API is unavailable?

> *Silent fallback to hardcoded mock findings. Each agent has realistic mock data (e.g., 'Key customer contracts contain change of control consent clauses' - critical). The architecture is production-ready; the intelligence is conditional on API key. This is a known gap -- users see 'completed' status even with mock data.*

Q: How does the financial import handle German accounting?

> *Main branch: 1) Detect structure by scanning 20 rows for headers/account codes/periods. 2) Detect SKR03 (revenue 4xxx) vs SKR04 (revenue 8xxx). 3) Parse German numbers ('1.234,56' -> 1234.56). 4) Map accounts to standardized categories. 5) UPSERT for monthly appends. V1.0 uses simpler pandas-based parsing.*

Q: What would you do differently if starting over?

> *1) Set up Alembic from day one -- create_all() doesn't handle schema changes. 2) Add tests early -- zero automated tests is a risk for financial calculations. 3) Wire Celery from the start -- BackgroundTasks are fragile. 4) Use one LLM provider consistently -- having both Vertex AI and Groq splits the codebase.*

Q: How would you scale for enterprise?

> *1) Celery + Redis for persistent task queues. 2) Cloud Storage instead of local disk. 3) Read replicas for PostgreSQL. 4) Parallel agent execution (LangGraph state graphs). 5) Cloud Run on GCP europe-west3 with auto-scaling.*

Q: What's the hardest technical problem you solved?

> *German number parsing for financial imports. German uses comma for decimals and dots for thousands ('1.234.567,89'). English is opposite ('1,234,567.89'). Parsing wrong corrupts every figure. We detect format by scanning sample cells -- if comma + exactly 2 digits at end, it's German. Then transform before Decimal().*

10. Numbers to Remember

Metric	Value	Context
Modules	11	All registered in main.py
API Endpoints	60+	Across all modules
Database Tables	17+	All modules combined
Docker Services	5	PostgreSQL, Redis, API, Celery, Web
User Roles	5	Admin, Lead, Team, Seller, Buyer
Agent Types	4	Planning, Legal, Tax, Finance
Planning Phases	5	Data -> Risk -> Dialog -> Plan -> List
Report Types	3	Detailed, Executive, Consolidated
Chunk Size	1500 chars	With 300 char overlap (20%)
Embedding Dim	768 (main)	text-embedding-004
FTS Top-K	10	Chunks per agent query
LLM (v1.0)	llama-3.3-70b	Groq API free tier
LLM (main)	gemini-1.5-pro	Vertex AI europe-west3
LLM Temperature	0.1-0.2	Low for consistency
JWT Access	60 minutes	HS256 algorithm
JWT Refresh	30 days	Session persistence
Max File	50 MB	Document upload limit
bcrypt Rounds	2^12 = 4096	~100ms per hash
SKR03 Revenue	4000-4999	German chart of accounts
SKR04 Revenue	8000-8999	Alternative chart
DD Request Items	15-40	Standard checklist
GCP Region	europe-west3	Frankfurt (EU only)
Est. Cloud Cost	EUR 100-250/mo	Cloud Run + SQL + Redis

Combined Completion Status

```
Module              Main    v1.0    Best
Auth + Security     100%    70%     Main (2FA, blacklist, GDPR)
Projects            100%    100%    Equal
DMS                  90%    75%     Main (versioning, search, tags)
OCR                  40%   100%     v1.0 (pymupdf, pytesseract)
AI Agents            45%    85%     v1.0 (Groq, strict prompts)
Planning            100%   100%     Main (40 items vs 15)
Finance             100%   100%     Main (SKR, German nums)
Reports               0%   100%     v1.0 (Word .docx)
Audit                 0%   100%     v1.0 (middleware)
Knowledge             0%    10%     Both scaffolded
Self-Improvement      0%    10%     Both scaffolded
Frontend             90%    95%     v1.0 (permissions, pages)
Alembic              Yes     No     Main

IDEAL: Merge both branches for ~90% spec coverage
```